

Exercícios Capacitação Python - Dia 4

Aquecimento

- 1) O que é uma função e quais são os benefícios de separar o nosso programa em múltiplas funções? Responda usando comentários ou um `print()`. Em seguida, crie uma função simples chamada `saudacao` usando a palavra reservada `def` que não recebe parâmetros e não possui valor de retorno. O corpo da função deve apenas utilizar a função `print()` para exibir "Olá, bem-vindo!". Fora da função, faça a chamada dela para testar a execução.
- 2) Defina uma função chamada `exibir_dobro` que receba um parâmetro `n` definindo seu tipo como `int` (`n: int`). A função não deve ter retorno de algum valor, mas deve calcular o dobro do valor recebido e imprimir o resultado. Chame a função passando um número previamente digitado pelo usuário via `input()`.
- 3) Crie uma função chamada `somar` que receba dois parâmetros: `a` e `b`, ambos do tipo `int`. A função deve ter o tipo de retorno definido explicitamente e devolver a soma dos dois valores informados, imprimindo na tela esse resultado.
- 4) Crie uma função `verificar_par` que receba um número inteiro. Usando o operador de módulo (%) visto nas aulas anteriores, verifique se o número é par. A função deve obrigatoriamente utilizar `return` para devolver um valor booleano (`True` se par, `False` se ímpar) ao invés de usar a função `print()`.
- 5) Em Python, existe um conceito chamado "escopo de variáveis", que garante que nenhuma variável criada dentro de uma função possa ser acessada fora dela. Para testar isso, crie uma função que declare uma variável chamada `mensagem` recebendo a string "Dentro da função" e a imprima. No seu código principal, tente criar uma variável com o exato mesmo nome recebendo "Fora da função". Chame a função e, logo depois, imprima a variável do programa principal para comprovar que elas são diferentes e independentes.

Avançando

- 6) Escreva uma função chamada `multiplicar_texto` que receba uma string `texto` e um inteiro `vezes`. A função deve utilizar `return` para devolver uma nova string que é o resultado da repetição do texto pela quantidade de vezes.
- 7) Crie uma função chamada `somar_lista` que receba uma lista de números inteiros como parâmetro (indicando `list[int]`). Dentro do escopo da função, crie uma variável inicializada em 0, utilize um laço `for` para percorrer e somar todos os elementos, e finalize imprimindo esse valor.
- 8) Defina uma função `classificar_idade` que recebe um número do tipo `int` representando a idade de uma pessoa. Usando estruturas condicionais combinadas (`if/elif/else`), a função deve processar e utilizar `return` para devolver uma string com o

texto: "Criança" (menor que 12), "Adolescente" (entre 12 e 17) ou "Adulto" (18 ou mais).

9) Crie uma função chamada `tabuada` que receba um inteiro `n`. Essa função não precisa retornar valores. O objetivo dela é encapsular a lógica de um laço `for` aliado ao `range(1, 11)` para calcular e imprimir instantaneamente a tabuada completa do parâmetro `n`.

10) Implemente uma função `calcular_desconto` que receba dois valores como parâmetro: o valor original de um produto e a porcentagem de desconto. A função deve calcular a subtração do desconto sobre o total, e retornar esse novo valor final atualizado para o chamador do código.

Desafios

11) A principal vantagem das funções é reduzir a quantidade de código repetido. Crie uma função `primo(num: int) -> bool`. A função deve retornar `True` se o parâmetro fornecido for primo e `False` caso encontre divisores.

12) Desenvolva uma função `filtrar_pares` que receba uma `list[int]`. A função deve instanciar uma nova lista vazia internamente. Use um laço de repetição `for` para avaliar cada elemento da lista fornecida como parâmetro; se o elemento for par (verificado por condicional), use o método `append()` para inseri-lo na lista local. O `return` da função deve ser esta nova lista.

13) Crie uma função `calculadora` que receba três parâmetros: dois números com ponto flutuante (`float`) e uma `string` contendo um símbolo de operação ("`+`", "`-`", "`*`", "`/`"). Utilize a estrutura avançada `match-case` para verificar qual operação foi enviada, execute a matemática correta entre os dois números e retorne o resultado. Para o caractere de divisão, inclua uma verificação preventiva para não dividir por zero.

14) Crie uma função chamada `inverter_string` que receba uma palavra. É estritamente proibido usar métodos nativos de inversão ou fatiamentos mágicos (`[::-1]`). Use um laço `for` ou `while`, acesse os índices de trás para frente, e utilize concatenação de strings para montar a palavra invertida passo a passo em uma variável interna. Retorne a string resultante.